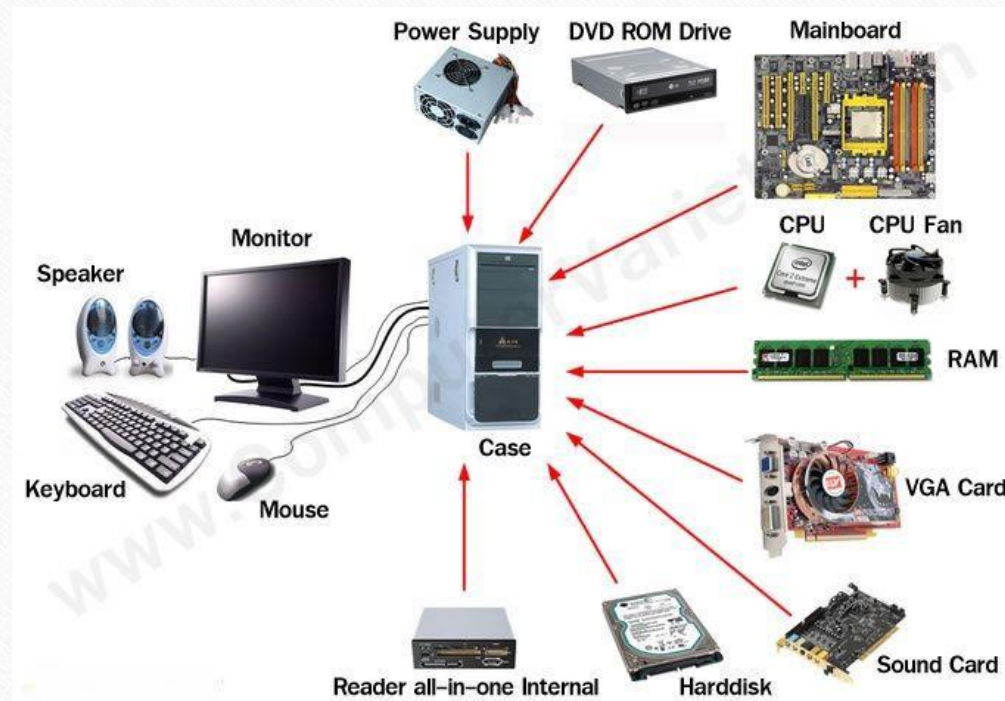


Programming in C

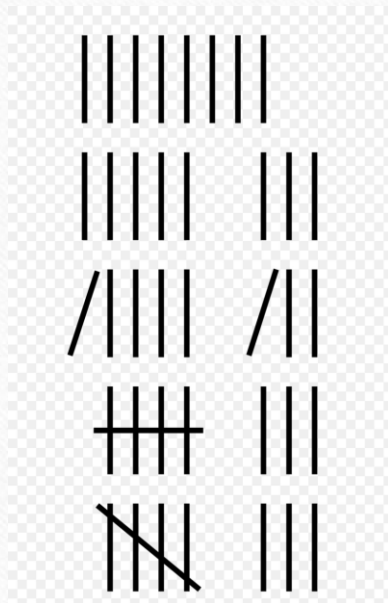
Hardware System



Number System

- Unary
- Binary
- Octal
- Decimal
- Hexadecimal

Unary Numeral System



Binary Number System

- 0 and 1
- Face value: 0 and 1
- Base/ Radix : 2
- Combinations to 0's and 1's lead to all possible number formation.
- 0,1,10,11,100,101,110,111,....

Octal Number System

- 0 -7
- Face value: 0,1,2,3,4,5,6,7
- Base/ Radix: 8
- Combinations to 0-7 leads to all possible number formation.
- 0,1,2,3,4,5,6,7,10,11,12,13,14,15,.....

Decimal Number System

- 0 -9
- Face value: 0,1,2,3,4,5,6,7,8,9
- Base/ Radix: 10
- Combinations to 0-9 leads to all possible number formation.
- 0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,.....

Hexadecimal Number System

- 0 -9 and A-F
- Face value: 0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F
- Base/ Radix: 16
- Combinations to 0-9 and A-F leads to all possible number formation.
- 0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F,10,11.....

Agenda

- Decimal to Other Base
- System Other Base System to Decimal
- Other Base System to Non-Decimal
- Shortcut method

Decimal to Other Base System

- There are the following steps that are used to convert the decimal number into a similar number of any base ' r '
- Step 1 – In the first step, we perform the division operation on integer and successive part with base ' r '. List down all the remainders till the quotient is zero. Then find out the remainders in reverse order
- Step 2 – The multiplication operation is done with base ' r ' of the fractional and successive until you get a resulting fractional part equal to zero.

Decimal to Binary conversion

- In the first step, we perform the division operation on the integer and the successive quotient with the base of binary(2).
- Next, we perform the multiplication on the integer and the successive quotient with the base of binary(2).
- Example 1: $(152.25)_{10}$

Operation	Quotient	Remainder
152/2	76	0 (LSB)
76/2	38	0
38/2	19	0
19/2	9	1
9/2	4	1
4/2	2	0
2/2	1	0
1/2	0	1(MSB)

Operation	Result	carry
0.25×2	0.50	0
0.50×2	0	1

10011000. 01

Decimal to Octal and hexadecimal

Decimal to Octal conversion

Operation	Quotient	Remainder
152/8	19	0
19/8	2	3
2/8	0	2

Operation	Result	carry
0.25×8	0	2

230.2

Decimal to hexadecimal conversion

Operation	Quotient	Remainder
152/16	9	8
9/16	0	9

Operation	Result	carry
0.25×16	0	4

98.4

Other Base System to Decimal

1. Binary to Decimal Conversion

- Step 1 – Determine the column positional value of each digit. This depends on the position of the digit and the base of the number system. (binary 4 2 1 where 4 is 2^2)
-
- Step 2 – Multiply the obtained column values in Step1 by the digits in the corresponding columns.

Example 1: $(10110.001)_2$

$$(10110.001)_2 = (1 \times 2^4) + (0 \times 2^3) + (1 \times 2^2) + (1 \times 2^1) + (0 \times 2^0) + (0 \times 2^{-1}) + (0 \times 2^{-2}) + (1 \times 2^{-3})$$

$$(10110.001)_2 = (1 \times 16) + (0 \times 8) + (1 \times 4) + (1 \times 2) + (0 \times 1) + (0 \times 1/2) + (0 \times 1/4) + (1 \times 1/8)$$

$$(10110.001)_2 = 16 + 0 + 4 + 2 + 0 + 0 + 0 + 0.125$$

$$(10110.001)_2 = (22.125)_{10}$$

Other Base System to Decimal

2. Octal to Decimal Conversion

1. Example 1: $(152.25)_8$

-
- $(152.25)_8 = (1 \times 8^2) + (5 \times 8^1) + (2 \times 8^0) + (2 \times 8^{-1}) + (5 \times 8^{-2})$
 $(152.25)_8 = 64 + 40 + 2 + (2 \times 1/8) + (5 \times 1/64)$
 $(152.25)_8 = 64 + 40 + 2 + 0.25 + 0.078125$
 $(152.25)_8 = 106.328125$

3. Hexa-decimal to Decimal Conversion

1. $(152A.25)_{16}$

- $(152A.25)_{16} = (1 \times 16^3) + (5 \times 16^2) + (2 \times 16^1) + (A \times 16^0) + (2 \times 16^{-1}) + (5 \times 16^{-2})$
 $(152A.25)_{16} = (1 \times 4096) + (5 \times 256) + (2 \times 16) + (10 \times 1) + (2 \times 16^{-1}) + (5 \times 16^{-2})$
 $(152A.25)_{16} = 4096 + 1280 + 32 + 10 + (2 \times 1/16) + (5 \times 1/256)$
 $(152A.25)_{16} = 5418 + 0.125 + 0.01953$
 $(152A.25)_{16} = 5418.14453125$

Other Base System to Non-Decimal

Step 1 – Convert the original number to a decimal number base10.

Step 2 – Convert the decimal number so obtained to the new base number.

Octal Number – 25_8 to Binary

Step 1 – Convert to Decimal

Octal Number	Decimal Number
25_8	$((2 \times 8^1) + (5 \times 8^0))_{10}$
25_8	$(16 + 5)_{10}$
25_8	21_{10}

Step 2 – Convert Decimal to Binary

Operation	Result	Remainder
$21 / 2$	10	1
$10 / 2$	5	0
$5 / 2$	2	1
$2 / 2$	1	0
$1 / 2$	0	1

ANS: 10101_2

Shortcut Method

Binary to Octal

1. Step 1: Make the pairs of three bits on both sides of the binary point. If there will be one or two bits left in a pair of three bits pair, we add the required number of zeros on extreme sides.
2. Step 2: Write the octal digits corresponding to each pair.

Example 1: $(111110101011.0011)_2$

111 110 101 011.001 100

7653.14

Shortcut Method

Binary to Hexadecimal Conversion

1. Step 1: Make the pairs of four bits on both sides of the binary point. If there will be one or two bits left in a pair of four bits pair, we add the required number of zeros on extreme sides.
2. Step 2: Write the hexadecimal digits corresponding to each pair.

Example 1: $(011110101011.0011)_2$

0111 1010 1011.0011

$(011110101011.0011)_2 = (7AB.3)_{16}$

Shortcut Method

Octal to Binary

- **Step 1** – Convert each octal digit to a 3 digit binary number (the octal digits may be treated as decimal for this conversion).
- **Step 2** – Combine all the resulting binary groups (of 3 digits each) into a single binary number.

Octal Number – 25_8

Calculating Binary Equivalent –

Octal Number	Binary Number
25_8	$2_{10} 5_{10}$
25_8	$010_2 101_2$
25_8	010101_2

Hexadecimal to Binary

Hexadecimal to Binary

- **Step 1** – Convert each hexadecimal digit to a 4 digit binary number (the hexadecimal digits may be treated as decimal for this conversion).
- **Step 2** – Combine all the resulting binary groups (of 4 digits each) into a single binary number.

Hexadecimal Number – 15_{16}

Calculating Binary Equivalent –

Hexadecimal Number	Binary Number
15_{16}	$1_{10} \ 5_{10}$
15_{16}	$0001_2 \ 0101_2$
15_{16}	00010101_2

Decimal to Binary Conversion

2)156

2)78

2)39

2)19

2)9

2)4

2)2

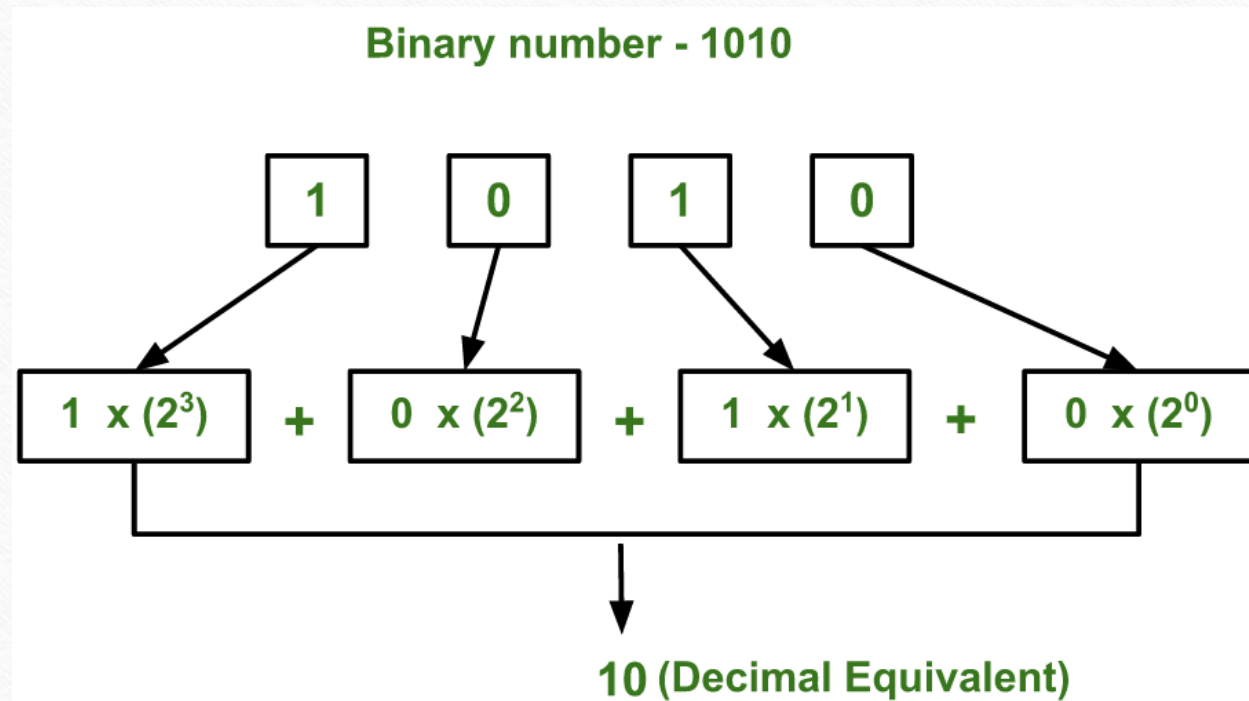
2)1

Remainder:

0
0
1
1
1
0
0
1

$$156_{10} = 10011100_2$$

Binary to Decimal Conversion



$$(1010)_2 = (10)_{10}$$

Binary to Decimal Conversion

128	64	32	16	8	4	2	1
1	0	0	1	1	1	0	0

$$156_{10} = 10011100_2$$

Why Computer uses Binary number system only and not any other NS?

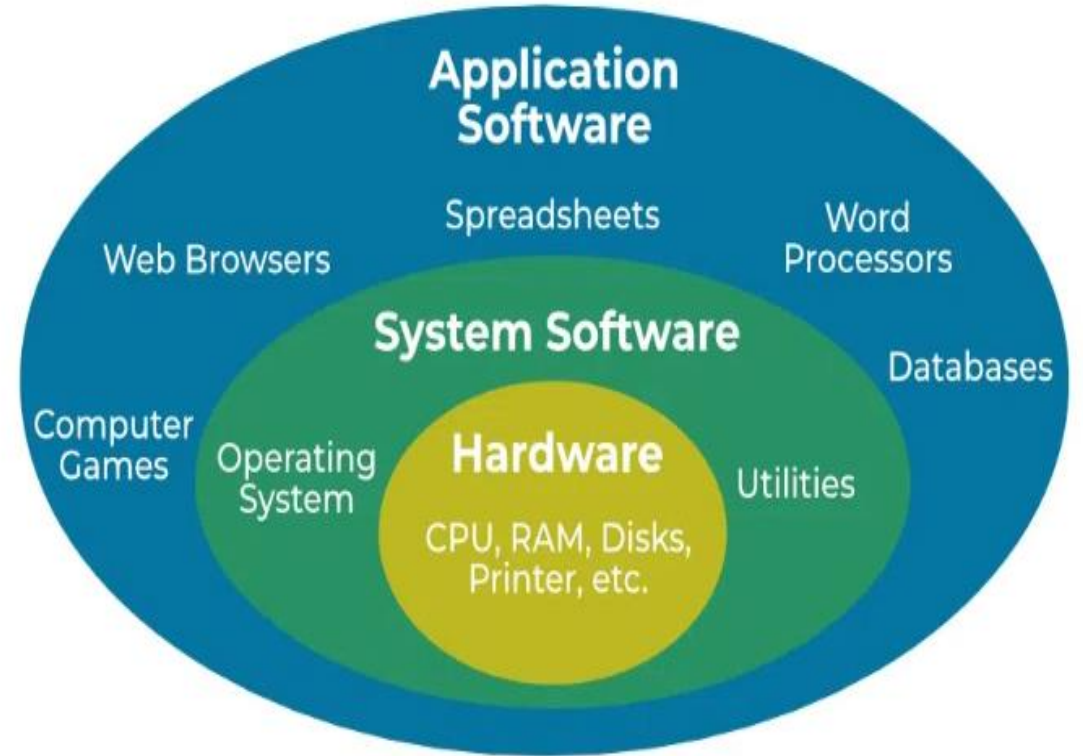
- Everything is represented by a binary electrical signal that registers in one of two states: on or off.

ASCII(American Standard Code for Information Interchange)

Dec	Bin	Hex	Char	Dec	Bin	Hex	Char	Dec	Bin	Hex	Char	Dec	Bin	Hex	Char
0	0000 0000	00	[NUL]	32	0010 0000	20	space	64	0100 0000	40	@	96	0110 0000	60	`
1	0000 0001	01	[SOH]	33	0010 0001	21	!	65	0100 0001	41	A	97	0110 0001	61	a
2	0000 0010	02	[STX]	34	0010 0010	22	"	66	0100 0010	42	B	98	0110 0010	62	b
3	0000 0011	03	[ETX]	35	0010 0011	23	#	67	0100 0011	43	C	99	0110 0011	63	c
4	0000 0100	04	[EOT]	36	0010 0100	24	\$	68	0100 0100	44	D	100	0110 0100	64	d
5	0000 0101	05	[ENQ]	37	0010 0101	25	%	69	0100 0101	45	E	101	0110 0101	65	e
6	0000 0110	06	[ACK]	38	0010 0110	26	&	70	0100 0110	46	F	102	0110 0110	66	f
7	0000 0111	07	[BEL]	39	0010 0111	27	'	71	0100 0111	47	G	103	0110 0111	67	g
8	0000 1000	08	[BS]	40	0010 1000	28	(	72	0100 1000	48	H	104	0110 1000	68	h
9	0000 1001	09	[TAB]	41	0010 1001	29	)	73	0100 1001	49	I	105	0110 1001	69	i
10	0000 1010	0A	[LF]	42	0010 1010	2A	*	74	0100 1010	4A	J	106	0110 1010	6A	j
11	0000 1011	0B	[VT]	43	0010 1011	2B	+	75	0100 1011	4B	K	107	0110 1011	6B	k
12	0000 1100	0C	[FF]	44	0010 1100	2C	,	76	0100 1100	4C	L	108	0110 1100	6C	l
13	0000 1101	0D	[CR]	45	0010 1101	2D	-	77	0100 1101	4D	M	109	0110 1101	6D	m
14	0000 1110	0E	[SO]	46	0010 1110	2E	.	78	0100 1110	4E	N	110	0110 1110	6E	n
15	0000 1111	0F	[SI]	47	0010 1111	2F	/	79	0100 1111	4F	O	111	0110 1111	6F	o
16	0001 0000	10	[DLE]	48	0011 0000	30	0	80	0101 0000	50	P	112	0111 0000	70	p
17	0001 0001	11	[DC1]	49	0011 0001	31	1	81	0101 0001	51	Q	113	0111 0001	71	q
18	0001 0010	12	[DC2]	50	0011 0010	32	2	82	0101 0010	52	R	114	0111 0010	72	r
19	0001 0011	13	[DC3]	51	0011 0011	33	3	83	0101 0011	53	S	115	0111 0011	73	s
20	0001 0100	14	[DC4]	52	0011 0100	34	4	84	0101 0100	54	T	116	0111 0100	74	t
21	0001 0101	15	[NAK]	53	0011 0101	35	5	85	0101 0101	55	U	117	0111 0101	75	u
22	0001 0110	16	[SYN]	54	0011 0110	36	6	86	0101 0110	56	V	118	0111 0110	76	v
23	0001 0111	17	[ETB]	55	0011 0111	37	7	87	0101 0111	57	W	119	0111 0111	77	w
24	0001 1000	18	[CAN]	56	0011 1000	38	8	88	0101 1000	58	X	120	0111 1000	78	x
25	0001 1001	19	[EM]	57	0011 1001	39	9	89	0101 1001	59	Y	121	0111 1001	79	y
26	0001 1010	1A	[SUB]	58	0011 1010	3A	:	90	0101 1010	5A	Z	122	0111 1010	7A	z
27	0001 1011	1B	[ESC]	59	0011 1011	3B	;	91	0101 1011	5B	[	123	0111 1011	7B	{
28	0001 1100	1C	[FS]	60	0011 1100	3C	<	92	0101 1100	5C	\	124	0111 1100	7C	
29	0001 1101	1D	[GS]	61	0011 1101	3D	=	93	0101 1101	5D	]	125	0111 1101	7D	}
30	0001 1110	1E	[RS]	62	0011 1110	3E	>	94	0101 1110	5E	^	126	0111 1110	7E	~
31	0001 1111	1F	[US]	63	0011 1111	3F	?	95	0101 1111	5F	_	127	0111 1111	7F	[DEL]

Software

- Software is a set of instructions or commands that tells a user how to do and what to do.
- Types of Software: System Software and Application Software
- System Software is that without which, system can not run. System Software maintain the system resources and give the path for application software to run.
- Application software is built for specific tasks. High-level languages are used to write the application software



IDE (Integrated Development Environment)

- IDE provides multiple facilities to programmers for software development.
- IDEs increase programmer productivity by combining common activities of writing software into a single application: editing source code, building executables, and debugging.
- The common features are: Editor, Compiler, linker, loader, debugger.

Compiler and Interpreter

- Compiler and interpreter are software programs that convert a high-level language into a machine language (0's and 1's binary form) that a computer can understand and perform tasks as per the program's instructions.
- But there are variations in the working process and steps of a compiler and interpreter. A compiler converts complete source code into machine code at once. And finally, the program get executed.
- Interpreter converts high-level programming language into machine language line-by-line while interpreting and running the program.

Linker, Loader and Debugger

- Linker generates the executable module of a source program. It takes as input, the object code generated by an assembler.
- Loader loads the executable module to the main memory. It takes executable module generated by a linker. It allocates the addresses to an executable module in main memory for execution.
- A debugger is a software program used to test and find bugs (errors) in other programs.

IDE Examples



Code::Blocks



Programming Languages

❑ Natural languages

- Interfaces for expressing information
 - ideas, knowledge, commands, questions, ...
 - Facilitate communication between people
- Different natural languages
 - English, Chinese, French, German, ...

❑ Programming languages

- Interfaces for expressing data and algorithms
 - Instructing machines what to do
 - Facilitate communication between computers and programmers
- Different programming languages
 - FORTRAN, Pascal, C, C++, Java, Lisp, Python, Prolog, ...

Types of Computer Language

Machine Level Language

- It is the lowest level programming language understood by computers only
- A manufacturer designs a computer to obey just one language, its machine code, which is represented inside the computer by a string of binary digits (bits) 0 and 1.

Assembly language

- These languages have same structure and set of commands as machine language but they enable a programmer to use names instead of numbers
- The instructions of the assembly language are converted to machine codes by a language translator and then they are executed by the computer.

High-Level Languages

- Any language that is independent of the machine
- Closer to human languages
- Programs written in this language are translated into machine language by compiler or interpreter
- Examples include FORTRAN, C++, Java, PROLOG...

Type of Language	Example Language	Description	Example Instructions
High-level Language	Python, Visual Basic, Java, C++	Independent of hardware (portable). Translated using either a compiler or interpreter. One statement translates into many machine code instructions.	payRate = 7.38 Hours = 37.5 Salary = payRate * Hours
Low-level Language	Assembly Language	Translated using an assembler. One statement translates into one machine code instruction.	LDA181 ADD93 STO185
	Machine Code	Executable binary code produced either by a compiler, interpreter or assembler.	101010001101010101001001010101

Role of Programming Languages

Toward high-level language

- Machine language is low level, full of details
- Assembly language is a variant of machine language in which names and symbols take the place of actual codes

Benefits of higher -level language

- Readability
- Machine independent (portable)
- Availability of program libraries
- Easy error-checking

Need to study Programming Languages

- To improve your ability to develop effective algorithms
- To improve your use of existing programming languages
- To increase your vocabulary of useful programming constructs
- To allow a better choice of programming language
- To make it easier to learn a new language
- To make it easier to design a new language

History of C Language

- C programming language was developed in 1972 by Dennis Ritchie at bell laboratories of AT&T (American Telephone & Telegraph), located in the U.S.A.
- **Dennis Ritchie** is known as the **founder of the C language**.
- It was developed to overcome the problems of previous languages such as B, BCPL, etc.
- Initially, C language was developed to be used in **UNIX operating system**. It inherits many features of previous languages such as B and BCPL.
- C is one of the middle-level language, C combines the features of both high-level and low-level languages. It can be used for low-level programming, such as scripting for drivers and kernels and it also supports functions of high-level programming languages, such as scripting for software applications etc.

Programming languages that were developed before C language

Language	Year	Developed By
Algol	1960	International Group
BCPL	1967	Martin Richard
B	1970	Ken Thompson
Traditional C	1972	Dennis Ritchie
K & R C	1978	Kernighan & Dennis Ritchie
ANSI C	1989	ANSI Committee
ANSI/ISO C	1990	ISO Committee
C99	1999	Standardization Committee

The C language has formed the basis for many languages including C++, C-, C#, Objective-C, BitC, C-shell, csh, D, Java, JavaScript, Go, Rust, Julia, Limbo, LPC, PHP, Python, Perl, Seed7, Vala, Verilog and many more other languages are there.

Features of 'C'

- HLL
- Simple Language (32 keywords)
- Structured Programming and Modular Language
- Efficient and Fast
- Extensible language/Ability to extend itself
- Case sensitive language
- Pointers
- Portable
- Embedded Systems (System Programming)
- Built in functions and operators
- Core language as other languages are dependent on it
- Dynamic memory allocation
- Statistically typed

Program Structure

- Documentation Section
- Link Section
- Definition Section
- Global Declaration Section
- Main() Function
- Subprogram Section

Basic Structure of a C Program

BASIC STRUCTURE OF A 'C' PROGRAM:

Documentation section [Used for Comments]
Link section
Definition section
Global declaration section [Variable used in more than one function]
main() { Declaration part Executable part }
Subprogram section [User-defined Function] Function1 Function 2 : : Function n

Example:

→ `//Sample Prog Created by:Bsource`

→ `#include<stdio.h>`
`#include<conio.h>`

→ `#define pi 3.14`

→ `int a=10;`

→ `void main()`
`{`
`clrscr();`
`printf("a value inside main(): %d",a);`
`fun();`
`}`

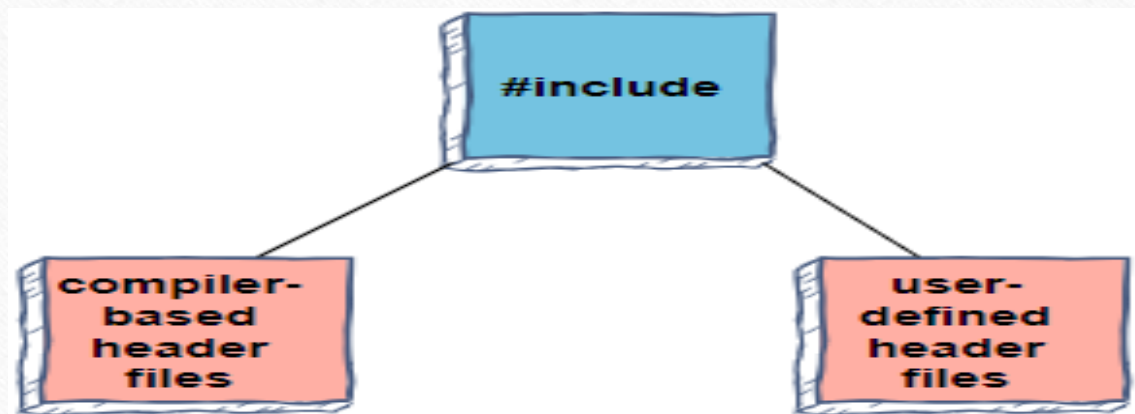
→ `void fun()`
`{`
`printf("\na value inside fun(): %d",a);`
`}`

Documentation Section :

- It has set of comment lines(name of program, author details).
- Comment line: To guide a programmer. It is a note for function, operation, logic in between a program.
- It is a Non-executable statement.
- / (single line comment)
- // (single line comment)
- /* hgjhghjg */ (multiple line comment)
- Can't be nested For Eg: /* SIT /* NAGPUR */ SIT */ is not allowed

Link Section:

- `#include` is called as pre-processor directive in C/C++ programming language. It is used to include a header file into your source code
- It provides instructions to the compiler to link function from the system library.
- `#include` directive is written at the beginning of a file to ensure that all header files are included before compiling the rest of the code. There are two ways that this directive can be used:



Link Section:

- **Including using <>:** It is mainly used to access pre-existing system header files located in the standard system directories (e.g. `#include<stdio.h>`)

While importing a file using angular brackets(<>), the preprocessor uses a predetermined directory path to access the file.

- **Including using " ":** This type is mainly used to access any header files of the user's program or user-defined files.

`#include "user-defined_file"`

When using the double quotes(" "), the preprocessor accesses the current directory in which the source "header_file" is located or the standard system directories.

Definition Section:

- It defines all symbolic constants.
- `#define` instruction defines value to a symbolic constant.
- `#define`:-
- It is a preprocessor compiler directive, not a statement.
- Therefore it should not end with a semicolon.
- Generally written in uppercase.

Global Declaration Section:

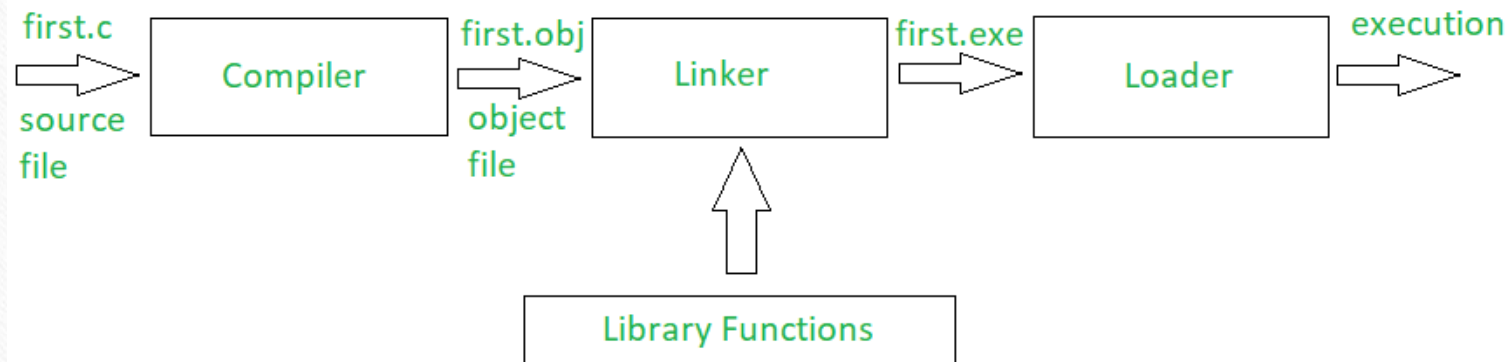
- Some variables that are used in more than one function, such variables are called as global variables.
- It is declared in the global declaration section.
- It also declares all the user-defined function.

Main() function section:

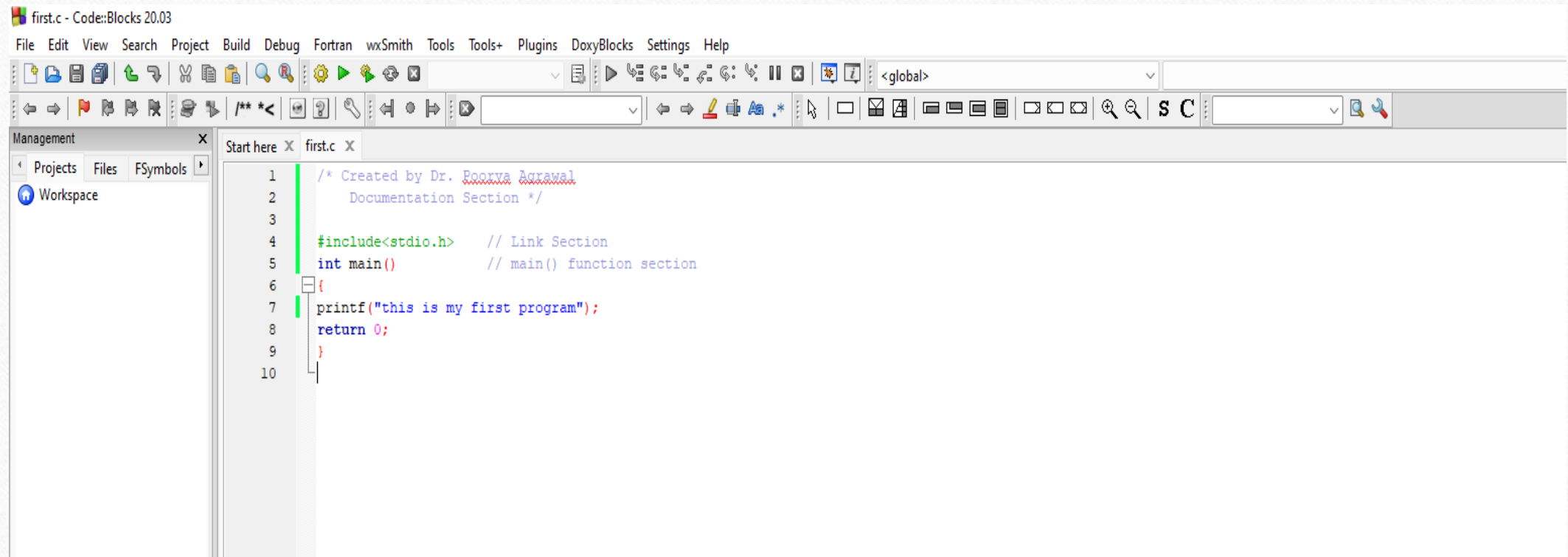
- Every 'C' program must have one main() function section.
- It contains two parts
- Declaration part: It declares all variables used in the executable part.
- Executable part: It has atleast one statement.

What happens after compiling and running

- A raw C program file is called Source file. It is saved with extension .c
- After compilation, the file is stored as the same name with extension .obj
- After execution, The linker links the object files to the library functions and the program becomes a .exe file extension with the same name.



First Program



The screenshot displays the Code::Blocks IDE interface. The title bar reads 'first.c - Code::Blocks 20.03'. The menu bar includes File, Edit, View, Search, Project, Build, Debug, Fortran, wxSmith, Tools, Tools+, Plugins, DoxyBlocks, Settings, and Help. The toolbar contains icons for file operations, editing, and execution. The left sidebar shows the 'Management' pane with 'Projects', 'Files', and 'FSymbols' tabs, and a 'Workspace' icon. The main editor window has tabs for 'Start here' and 'first.c'. The code in 'first.c' is as follows:

```
1  /* Created by Dr. Poorva Agrawal
2     Documentation Section */
3
4  #include<stdio.h>    // Link Section
5  int main()           // main() function section
6  {
7      printf("this is my first program");
8      return 0;
9  }
10
```


C Tokens

- A token in C can be defined as the smallest individual element of the C programming language that is meaningful to the compiler. It is the basic component of a C program.
- The tokens of C language can be classified into six types based on the functions they are used to perform.



Keywords

- Words whose meaning are explained to the C Compiler before.
- Also known as reserved words.
- 32 keywords

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

Identifiers

- Identifiers are used as the general terminology for the naming of variables, functions, and arrays. These are user-defined names consisting of an arbitrarily long sequence of letters and digits with either a letter or the underscore(_) as a first character. Identifier names must differ in spelling and case from any keywords.
- Rules for Naming Identifiers:
 - They must begin with a letter or underscore(_).
 - They must consist of only letters, digits, or underscore. No other special character is allowed.
 - It should not be a keyword.
 - It must not contain white space.
 - It should be up to 31 characters long as only the first 31 characters are significant.

Constants

- Constant: It is a name given to a variable whose value cannot be changed.
- Constants include: Primary and Secondary Constants
- Primary Constants are:
 - Integer Constant: at least one digit, no decimal point, either positive or negative
 - Real Constant: it must have at least one digit, decimal point, either positive or negative
 - Character Constant: A character constant is a single alphabet, a single digit or a single special symbol enclosed within single inverted commas. The maximum length of a character constant can be 1 character.
- Secondary Constants are:
 - Array, Pointer, Structure etc.

Constants

- Two ways to use constants:
- `#define`
 - For eg: `#define bp_water 100` Here, Memory is not allocated in RAM. It is just a substitution or replacement variable.
- `const`
 - For eg: `const int x=5;` Here, we have created a constant x whose value assigned in '5'. The value assigned to constant is a literal.

String

- It is a collection of characters in a linear sequence.
- It is terminated by a null character '\0'.
- String Declaration:
 - 1. Syntax: `char variableName[number of characters];`
For eg: `char name[6];`
 - 2. Pointers: `char *name;`
- String Initialization:
 - `char name[6] = {'I', 'N', 'D', 'I', 'A', '\0'};`
 - `char name[] = 'INDIA';` It will need 6 byte of space

Special Symbols

- **Brackets[]**: Opening and closing brackets are used as array element references. These indicate single and multidimensional subscripts.
- **Parentheses()**: These special symbols are used to indicate function calls and function parameters.
- **Braces{ }**: These opening and ending curly braces mark the start and end of a block of code containing more than one executable statement.
- **Comma (,)**: It is used to separate more than one statement like for separating parameters in function calls.
- **Colon(:)**: It is an operator that essentially invokes something called an initialization list.
- **Semicolon(;)**: It is known as a statement terminator. It indicates the end of one logical entity. That's why each individual statement must be ended with a semicolon.
- **Asterisk (*)**: It is used to create a pointer variable and for the multiplication of variables.
- **Assignment operator(=)**: It is used to assign values and for logical operation validation.
- **Pre-processor (#)**: The preprocessor is a macro processor that is used automatically by the compiler to transform your program before actual compilation.
- **Period (.)**: Used to access members of a structure or union.
- **Tilde(~)**: Used as a destructor to free some space from memory.

Operators

- C Programming is rich in built in operators. Operators are symbols that performs an action when applied to C variables and other objects. Operators act on operands.
- C operators are used to perform operations.
- Operators has arity, priority and associativity associated with them.
- Operators can be identified as:
 - Unary: It needs single operand to act upon.
 - Binary: It needs two operands to act upon.
 - Ternary: It needs three operands to act upon.

Types of Operators

- Arithmetic Operators (+, -, *, /, %)
- Relational Operators (<, >, <=, >=, !=, ==)
- Logical Operators (&&, ||, !)
- Assignment Operator (=, +=, -=, *=, /=, %=)
- Post and Pre Increment/Decrement Operator (++ , --)
- Conditional Operator (?:)
- Bitwise Operator (&, |, ^, ~, <<, >>)
- Special operators (, and sizeof)

	Operators	Type
Unary Operator →	++, --	Unary Operator
Binary Operator {	+, -, *, /, %	Arithmetic Operator
	<, <=, >, >=, ==, !=	Relational Operator
	&&, , !	Logical Operator
	&, , <<, >>, ~, ^	Bitwise Operator
	=, +=, -=, *=, /=, %=	Assignment Operator
Ternary Operator →	?:	Ternary or Conditional Operator

Variables

- Variables: An entity whose value vary. Variable names are names given to locations in memory. These locations can contain integer, real or character constants.
- $x=10$, $x=20$

Rules for Constructing Variable Names

- A variable name is any combination of 1 to 31 alphabets, digits or underscores.
- The first character in the variable name must be an alphabet or underscore.
- No commas or blanks are allowed within a variable name.
- No special symbol other than an underscore can be used in a variable name.
- It is always advisable to construct meaningful variable names.
- For Eg: si_int, prin,roi etc

Variables

- They are of two types:
- Initialized variables: Auto initialized to 0. As the memory location is allotted to a, its values becomes 0.
 - Global Variable
 - Static Variable For eg: `static int a;`
- Uninitialized Variables: Contains garbage value, local variable

Algorithm

Logical step by step method to solve a problem.

It is a procedure to solve a problem.

Algorithm is the first step of the process.

Used to solve a mathematical or computer problem.

Algorithm includes calculations reasoning and data processing.

An algorithm is defined as a well-defined sequence of steps that provides a solution for a given problem, whereas a **pseudocode** is one of the methods that can be used to represent an algorithm.

Example of an Algorithm

- Write an algorithm for adding three numbers
- Step 1: Start
- Step 2: Read num1, num2, num3
- Step 3: $\text{Result} = \text{num1} + \text{num2} + \text{num3}$
- Step 4: Print Result
- Step 5: End

Characteristics and Advantages

Characteristics :

- Clear and Unambiguous : Steps, input/output must be clear and must lead to one meaning.
- Finiteness : Algorithm must terminate after finite number of steps.
- Feasibility: Should be feasible with available resources
- Independent: Algorithm must be independent of any programming code

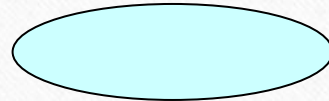
Advantages:

- It is easy to understand even a complex program
- It does not follow C programming language syntax
- C programs can be easily generated by pseudo-code
- It allows us to understand the logic of a program very quickly
- Pseudo-code can be modified easily
- Helps in correcting the error.

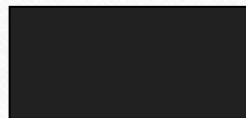
Flowchart

- A graphical or pictorial representation of an algorithm with the help of different symbols, shapes and arrows to demonstrate a process or a program.
- The main purpose of flowchart is to analyze different methods.
- Represents the flow of program
- Symbols:-

Start-Begin/End



Process box/operation box



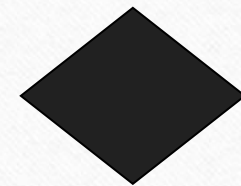
Connector



Input/Output Rectangle

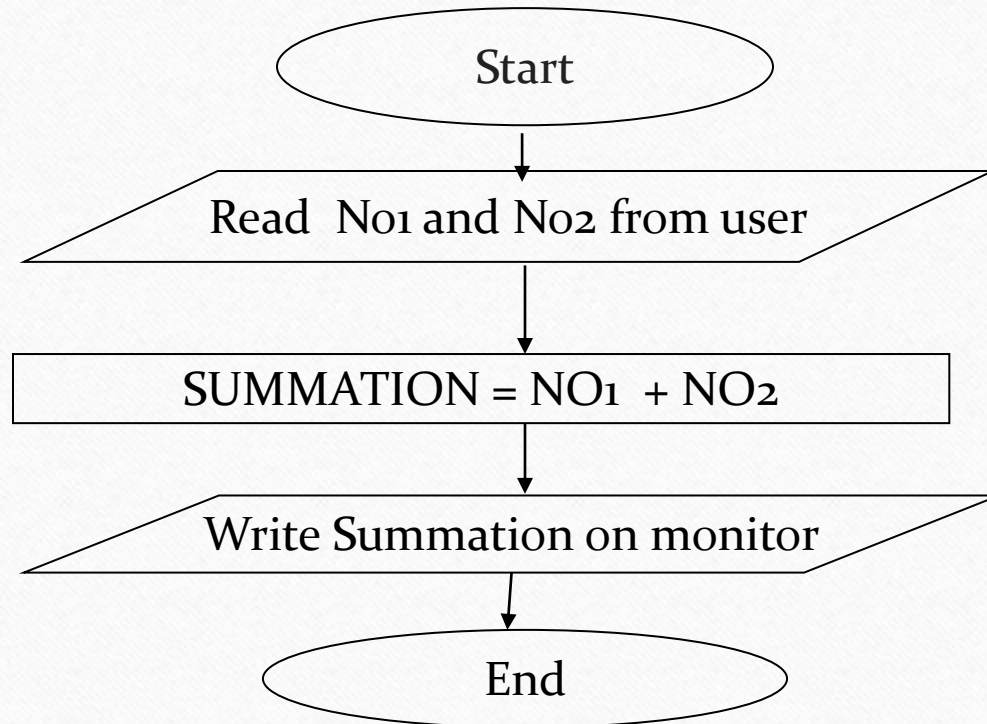


Decision Box



Flowchart

- Example: To add two numbers and display the result.

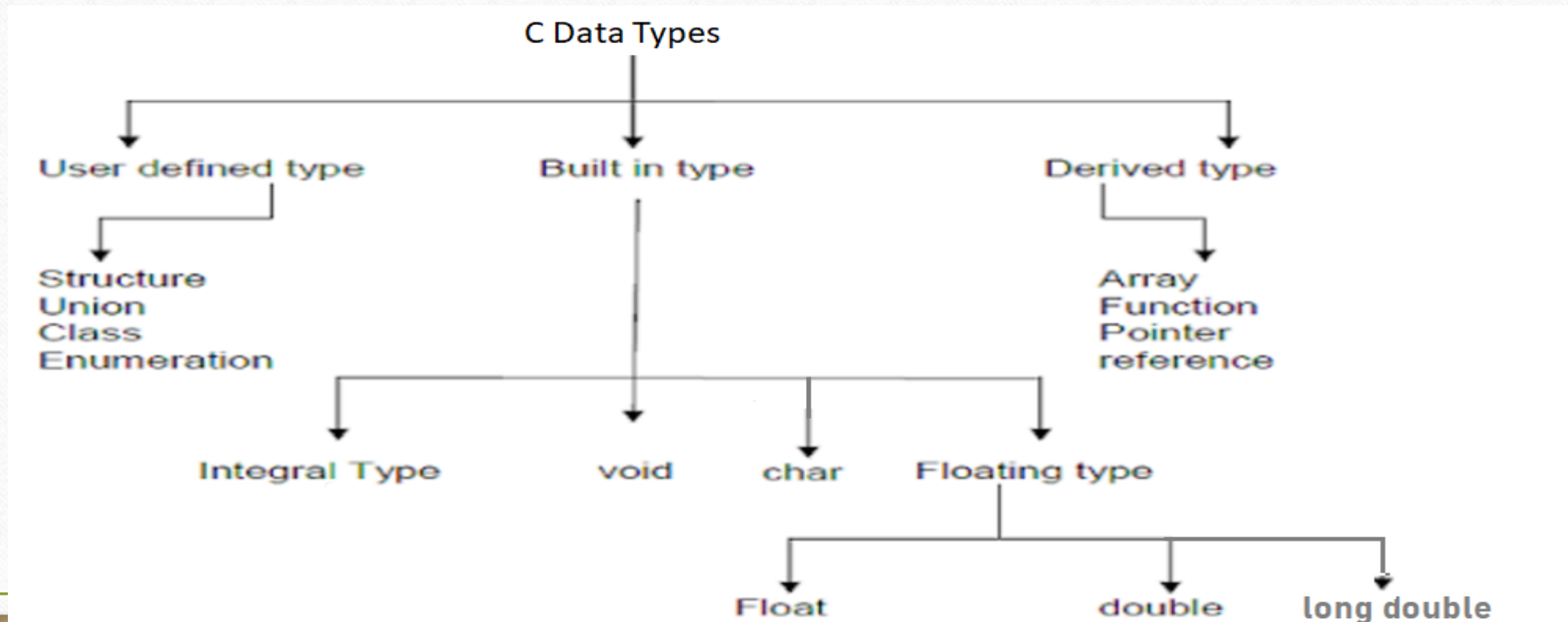


Problem Statements on Algorithm and Flowchart

- **Design an algorithm and flowchart for the following:**
- Compute the average of three numbers.
- Determine whether the number given by the user is odd or even.
- Find the largest among three numbers given by the user.

Data Types

- Data types are means to identify the type of data and associated operations of handling it.
- Data types specifies the range of values the variable holds.
- Data types occupy different space in memory depending on the type of data and compiler.
- C provides various types of data-types which allow the programmer to select the appropriate type for the variable to set its value



Data type modifiers in C

- Data Type Modifiers are keywords used to change the properties of current properties of data type. Data type modifiers are classified into following types:
 - long
 - short
 - unsigned
 - signed
- Modifiers long and short are prefixed with basic data types to modify (either increase or decrease) the amount of storage space allocated to a variable.
- For example, storage space for int data type is 4 byte for 32 bit processor. We can increase the range by using long int which is 8 byte. We can decrease the range by using short int which is 2 byte.

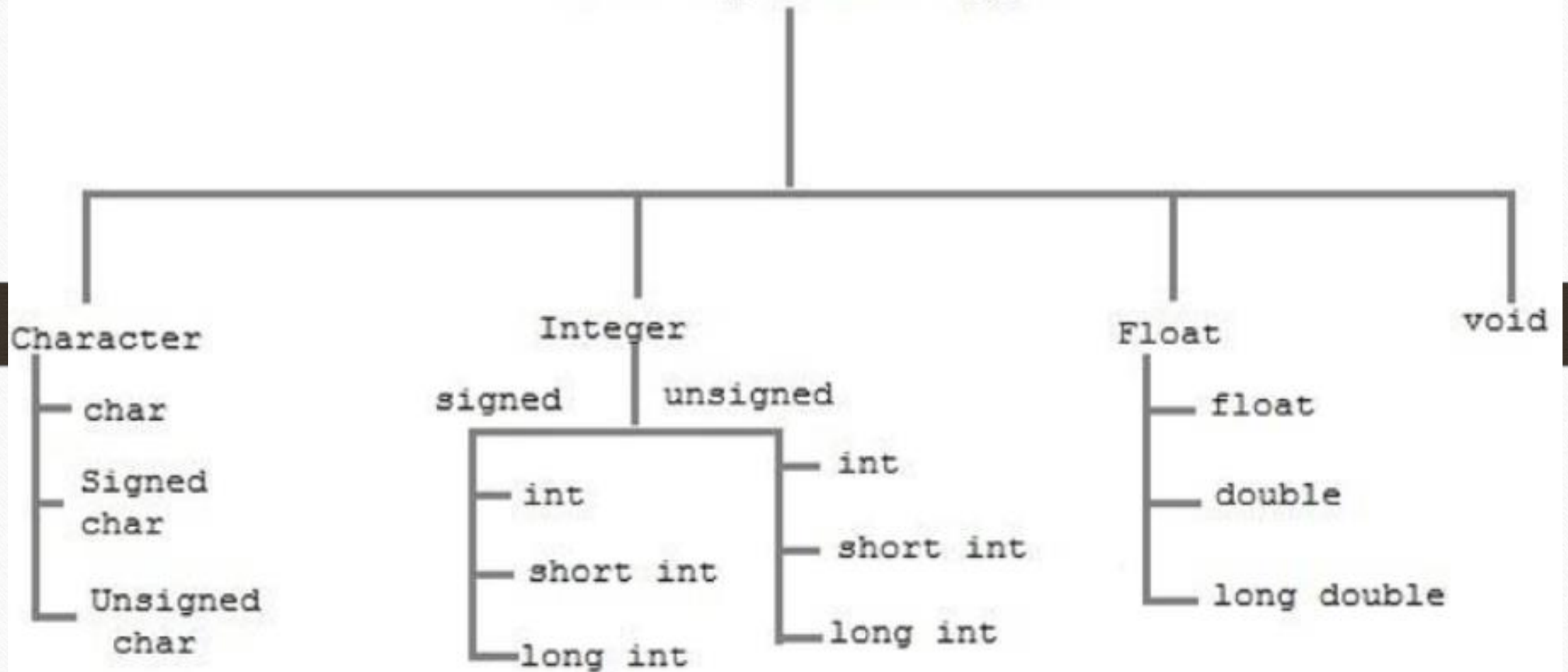


- **Unsigned-** This keyword can be used to make the accepting values of a data type is positive data type.
- **Signed-** This keyword accepts both negative or positive value and this is default properties or data type modifiers for every data type.

```
unsigned int a =100; // right
unsigned int a=-100; // wrong
```

```
int a=10;      // right
int a=-10;     // right
signed int a=10; // right
signed int a=-10; // right
```

Primary Data type



Storage sizes and value ranges of Built-in or Primary storage data type

Variable Type	Keyword	Bytes Required	Range
Character	char	1	-128 to 127
Unsigned character	unsigned char	1	0 to 255
Integer	int	2 or 4 bytes	-32768 to 32767
Short Integer	short int	2	-32768 to 32767
Long Integer	long int	4	-2,147,483,648 to 2,147,438,647
Unsigned Integer	unsigned int	2	0 to 65535
Unsigned Short integer	unsigned short int	2	0 to 65535
Unsigned Long Integer	unsigned long int	4	0 to 4,294,967,295

Type	Storage size	Value range	Precision
float	4 byte	1.2E-38 to 3.4E+38	6 decimal places
double	8 byte	2.3E-308 to 1.7E+308	15 decimal places
long double	10 byte	3.4E-4932 to 1.1E+4932	19 decimal places

- To get the exact size of a type or a variable on a particular platform, you can use the **sizeof** operator. The expressions *sizeof(type)* yields the storage size of the object or type in bytes.

```
#include <stdio.h>
int main() {
    short a;
    long b;
    long long c;
    double d;
    printf("size of short = %d bytes\n", sizeof(a));
    printf("size of long = %d bytes\n", sizeof(b));
    printf("size of long long = %d bytes\n", sizeof(c));
    printf("size of double= %d bytes\n", sizeof(d));
    return 0;
}
```

```
size of short = 2 bytes
size of long = 4 bytes
size of long long = 8 bytes
size of double= 8 bytes
```

Derived Data Types

- The data-types that are derived from the primitive or built-in datatypes are referred to as Derived Data Types. These can be of four types namely:

- Array
- Function
- Pointers
- References

Type Conversion

- C/C++ allows us to convert data of one type to that of another. This is known as type conversion.

- There are two types of type conversion:

- Implicit Conversion (also known as automatic type conversion)
- Explicit Conversion (also known as Type Casting)

- **Implicit type conversion:** It is the type conversion that is done automatically done by the compiler. This occurs when an expression contains variables of more than one type. All the data types present in the expression are converted to data type of the variable with the largest data type.

- Below is the order of the automatic type conversion:

bool -> char -> short int -> int -> unsigned int -> long -> unsigned -> long long -> float -> double -> long double

- Implicit conversions can lose information such as signs can be lost when signed type is implicitly converted to unsigned type

- We cannot perform implicit type casting on the data types which are not compatible with each other such as:

- Converting float to an int will truncate the fraction part hence losing the meaning of the value.
- Converting double to float will round up the digits.
- Converting long int to int will cause dropping of excess high order bits.

```
#include <stdio.h>
main()
{
    int int1 = 100;
    char char1 = 'c';
    // char1 implicitly converted to int using ASCII value of 'c' i.e. 99
    int1 = int1 + char1;
    // int1 is implicitly converted to float
    float flt1 = int1 + 2.7;
    printf ("int1 : %d \n", int1);
    printf ("char1 : %c \n", char1);
    printf ("flt1 : %f \n", flt1);
}
```

Output:

```
int1 : 199
char1 : c
flt1 : 201.699997
```

Type Conversion

- **Explicit conversion or type casting:** It is user defined type conversion, the user converts one type of variable to another type. It is considered as forceful casting
- If type conversion is not used the loss of data occurs. Example:

```
int result, var1=5, var2=2;
```

```
result=var1/var2;
```

output: 2 not 2.5

2.5



will be lost

- To force the type conversion in such situations, we use explicit type casting. It requires a type casting operator. The general syntax for type casting operations is as follows:

(type-name) expression

The type name is the standard 'C' language data type.

An expression can be a constant, a variable or an actual expression.

```
#include<stdio.h>
main()
{
    float a = 1.2;
    int b = (int)a + 1;
    printf("Value of a is %f\n", a);
    printf("Value of b is %d\n", b);
}
```

Output:

```
Value of a is 1.200000
Value of b is 2
```

Type Conversion

- **Examples of Explicit conversion or type casting:**

- ```
int female_no, male_no;
float ratio;
ratio=(float)female_no/male_no;
```

female\_no and male\_no will remain int in other parts of the program

- Casting can be used to round off the value:  

```
x = (int) 7.5; // 7.5 is converted to int
```
- ```
a= (int) 21.3 / (int) 4.5;  // this is evaluated as 21/4 and result is 5
```
- ```
y = (int) (a+b); // result of a+b is converted to int
```
- ```
z=(int) a + b;            // a is converted to int and then added to b
```


Operators and Expressions

- The symbols which are used to perform logical and mathematical operations in a program are called operators.
- These operators join individual constants and variables to form expressions.
- Operators, constants and variables, etc are combined together to form expressions.
- Consider the expression $A + B * 5$. where, $+$, $*$ are operators, A , B are variables, 5 is constant and $A + B * 5$ is an expression.

Operators

- An operator is a symbol that tells the compiler to perform specific mathematical or logical functions. C language is rich in built-in operators and provides the following types of operators:
 - Arithmetic operators
 - Assignment operators
 - Relational operators
 - Logical operators
 - Bit wise operators
 - Conditional operators (ternary operators)
 - Increment/decrement operators
 - Special operators

Operators

Types of Operators	Description
Arithmetic operators	These are used to perform mathematical calculations: +, -, *, /, %
Assignment operators	These are used to assign the values to the variables, result of an expression to a variable. Example: x+=5 (or x=x+5), a=a*(n+1) (or a*=n+1)
Relational operators	These operators are used to compare the value of two variables: <, <=, >, >=, ==, !=
Logical operators	These operators are used to perform logical operations on the given two variables &&, , ! Example: a>b&& x==10 //expression like this combining 2 or more relational expressions is called logical expression or compound relational expression
Increment/decrement operators (++ and --)	These operators are used to either increase or decrease the value of the variable by one. ++m and m++ are same if they form the statements independently and are different when they are used on the right hand side of an assignment statement Example: m=5; m=5; y=++m; y=m++; Value of y and m is 6 Value of y is 5 and m is 6

Operators

Types of Operators	Description
Bit wise operators & Bitwise AND Bitwise OR ^ Bitwise Exclusive OR << Shift Left >> Shift Right ~ Bitwise NOT (One’s Complement)	These operators are used to perform manipulation of data at bit level. These operators are used for testing the bits or shifting them right or left. Bitwise operators can’t be applied to float or double. Left shifting an integer “x” with an integer “y” (x<y) is equivalent to multiplying x with 2^y (2 raise to power y). Right shifting (x>y) is equivalent to dividing x with 2^y.
Conditional operator: ternary operator (?:)	Conditional operators return one value if condition is true and returns another value is condition is false. Exp1 ?: Exp2: Exp3; a=10; if(a>b) b= 15; or x=a; x= (a>b)? a: b; else x=b;
Special operators	sizeof() operator, member selection operators (., ->), comma operator, pointer operators (&, *)

C Program to demonstrate use of Bitwise Operators

```
#include <stdio.h>

int main()
{
    int a=5,b=9;
    printf("a=%d, b=%d\n",a,b);
    printf("a&b=%d\n",a&b);
    printf("a|b=%d\n",a|b);
    printf("a^b=%d\n",a^b);
    printf("~a=%d\n",~a);
    printf("b<<1=%d\n",b<<1);
    printf("b>>1=%d\n",b>>1);
    return 0;
}
```

```
a=5, b=9
a&b=1
a|b=13
a^b=12
~a=-6
b<<1=18
b>>1=4
```

a = 00000101
b = 00001001
a&b=00000001
a|b= 00001101

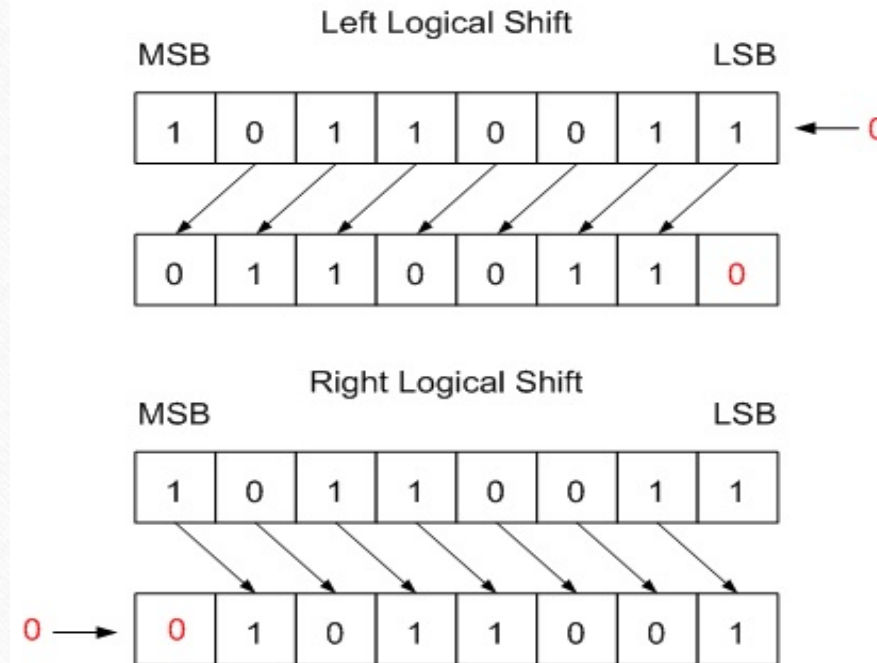


Fig. 1 Logical Shift by one bit

Precedence of operators

- The precedence of operators determines which operator is executed first if there is more than one operator in an expression.

Precedence	Operator	Description	Associativity
1	()	Parentheses (function call)	Left-to-Right
	[]	Array Subscript (Square Brackets)	
	.	Dot Operator	
	->	Structure Pointer Operator	
	++, —	Postfix increment, decrement	

Precedence of operators

Precedence	Operator	Description	Associativity
2	++ / --	Prefix increment, decrement	Right-to-Left
	+ / -	Unary plus, minus	
	! , ~	Logical NOT, Bitwise complement	
	(type)	Cast Operator	
	*	Dereference Operator	
	&	Addressof Operator	
	sizeof	Determine size in bytes	
3	*,/,%	Multiplication, division, modulus	Left-to-Right
4	+/-	Addition, subtraction	Left-to-Right
5	<< , >>	Bitwise shift left, Bitwise shift right	Left-to-Right

Precedence of operators

Precedence	Operator	Description	Associativity
6	< , <=	Relational less than, less than or equal to	Left-to-Right
	> , >=	Relational greater than, greater than or equal to	
7	== , !=	Relational is equal to, is not equal to	Left-to-Right
8	&	Bitwise AND	Left-to-Right
9	^	Bitwise exclusive OR	Left-to-Right
10		Bitwise inclusive OR	Left-to-Right
11	&&	Logical AND	Left-to-Right
12		Logical OR	Left-to-Right
13	?:	Ternary conditional	Right-to-Left

Precedence of operators

Precedence	Operator	Description	Associativity
14	=	Assignment	Right-to-Left
	+= , -=	Addition, subtraction assignment	
	*= , /=	Multiplication, division assignment	
	%= , &=	Modulus, bitwise AND assignment	
	^= , =	Bitwise exclusive, inclusive OR assignment	
	<<=, >>=	Bitwise shift left, right assignment	
15	,	comma (expression separator)	Left-to-Right

Basic Output Statements

- These statements are defined in the header file named: `stdio.h`
- `putchar()`: It is used to write a character in the output screen.
- `printf()` is an output function, it is used to display data on the screen.
 - SYNTAX: `printf("Message to be displayed FormatSpecifier", var_name);`
 - Eg: `printf("Welcome to SIT");` Welcome to SIT
 - `printf("Welcome to SIT\n Nagpur");` Welcome to SIT
Nagpur
 - `int num=5;`
 - `printf("The number is %d",num);` The number is 5

Format Specifier/Type Modifier

Format specifiers define the type of data to be printed on standard output.

Data type	Format Specifier
signed int	%d or %i
unsigned int	%u
long signed int	%ld
long unsigned int	%lu or %ul
float	%f

Data type	Format Specifier
char	%c
string	%s
double	%lf
octal	%o
hexadecimal	%x or %X

Escape Sequences

- It is a sequence of characters that doesn't represent itself when used inside string literal or character.
- It is composed of two or more characters starting with backslash \.

Escape Sequence	Meaning
\n	New Line
\a	Alarm or beep
\t	Tab (Horizontal)
\v	Vertical Tab
\f	Form Feed
\b	Backspace

Escape Sequence	Meaning
\\	Backslash
\'	Single Quote
\"	Double Quote
\?	Question Mark
\0	Null
\r	Carriage return

Basic Input Statement

- These statements are defined in the header file named: `stdio.h`
- `scanf()` is one of the input function, it is used to get input data from the user.
 - SYNTAX: `scanf("FormatSpecifier", &var_name);`

- Eg:

```
printf("Enter two numbers:");  
scanf("%d %d",&a,&b);
```

```
Enter two numbers:2  
7
```

```
printf("Enter your name\n");  
scanf("%s",&name);  
printf("Enter the name %[^\\n]s",&name);
```

```
Enter your name  
Poorva  
Poorva Agrawal
```